深 圳 大 学

《专业综合设计》项目汇报

题目: **GO2W 四足机器人 LiDAR 与可见光相机协同感知可视化系统设计与实现**

姓名: 李瑞宇

专业: 电子信息工程

学院: 电子与信息工程

学号: **2023280491**

指导教师: 赵 博

2026 年 6 月 30 日

目录

1. 项目背景与需求分析 3
1.1 项目背景 3
1.2 需求分析 3
1.3 技术挑战 3
2. 系统总体架构 4
3. 关键技术与实现过程 4
3.1 ROS2 数据包解析 4
3.2 时间戳同步算法 4
3.3 多帧点云累积策略 5
3.4 坐标系归一化与着色映射 6
3.4 可视化渲染与视频合成 6
4. 实验数据与结果分析 7
4.1 数据集概况 7
4.2 时间同步质量分析 7
4.3 特征分布对比 7
4.4 可视化结果展示 7
5. 结论与展望 9

GO2W 四足机器人 LiDAR 与可见光相机协同感知可视化系统设计与实现

电子与信息工程学院（电子信息工程） 李瑞宇

学号：2023280491

【摘要】本项目基于搭载激光雷达（LiDAR）和可见光相机的 GO2W 四足机器人，围绕多传感器协同感知这一核心命题，设计并实现了一套离线数据采集、时间同步对齐、点云预处理与可视化的完整流水线系统。系统从 ROS2 数据包（Bag）中提取点云帧与视频帧，以帧间最近邻时间戳匹配策略实现跨传感器时间同步，并采用多帧点云累积方案将稀疏单帧点云增密至约 6 万点的高密度场景表示，再经坐标系归一化、高度着色映射与 PIL 图像渲染，输出直观易读的同步对比视频。最终的可视化结果以左视频、右俯视图+侧视图的三分屏布局呈现，俯视图坐标方向与相机视野方向严格一致，有效实现了激光雷达点云与光学视频在空间语义层面的相互印证。

【关键词】协同感知；激光雷达；四足机器人；时间同步；点云可视化；ROS2

1. 项目背景与需求分析

1.1 项目背景

随着具身智能与自主移动机器人技术的快速发展，四足机器人在复杂非结构化环境下的感知能力正成为研究热点。激光雷达（LiDAR）以其高精度的三维测距能力见长，而可见光相机则能提供丰富的语义纹理信息，两者的互补融合，即协同感知，是实现机器人精确环境理解的重要技术路径。

本项目以深圳大学实验室配备的 GO2W 四足机器人为实验平台。该机器人搭载激光雷达（40 线，最大探测距离 200m，点频约 1.152M pts/s）与一台前置可见光相机（分辨率 1280×720，帧率约 14.3fps），传感器数据通过 ROS2 消息机制实时发布并记录。数据采集场所为校园实验场地，采集时间为 2026 年 06 月 10 日，共获得两段有效 ROS2 Bag 数据：bag_094554（约 19 秒）与 bag_102458（约 187 秒）。

1.2 需求分析

(1) 数据解析：从 ROS2 Bag 中准确提取激光雷达点云帧与视频帧，正确处理各自的时间戳；

(2) 时间同步：由于两路传感器采样频率不同（LiDAR 约 5Hz，相机约 14fps），需要建立可靠的跨传感器帧级时间对齐映射；

(3) 场景重建：单帧点云密度不足（仅约 2000–4000 点），难以辨别场景结构，需要设计点云增密方案；

(4) 可视化输出：以直观易读的方式将激光雷达感知结果与视频进行对比展示，做到"点云与视频的相互印证"。

1.3 技术挑战

(1) 相机标定缺失：两传感器之间的精确外参及相机内参矩阵未提供，限制了点云到像素平面的精确重投影，但不影响空间拓扑层面的协同感知可视化。

(2) 时间戳异步：LiDAR 与相机时间戳来自不同内部时钟，存在系统性偏移与抖动，需要基于最小绝对差策略构建帧级对齐映射。

(3) 点云稀疏性：单帧 40 线 LiDAR 在距离较远时点云密度急剧下降，仅依靠单帧难以识别场景中的障碍物，需要设计滑动窗口累积方案。

2. 系统总体架构

如表 1 所示，整个系统分为四个功能模块：数据解析模块、时间同步模块、点云处理模块与可视化渲染模块，处理结果最终输出为 MP4 同步对比视频。

系统整体采用全离线批处理架构，以 Python 3.13 为开发语言，核心依赖包括 NumPy、Pillow、imageio / imageio-ffmpeg。所有计算在宿主机 CPU 上串行执行，单帧渲染耗时约 0.1s，满足离线分析需求。

表 1 系统模块划分与主要功能

模块名称	主要功能	核心技术
数据解析	ROS2 Bag 读取、点云 PointCloud2 解包、视频帧提取	struct.unpack, ffmpeg/imageio
时间同步	构建 LiDAR 帧与视频帧的最近邻时间戳映射	最小绝对差匹配，JSON meta 存储
点云处理	多帧累积、地面滤除、坐标归一化、高度/强度着色	NumPy 向量化运算，滑动窗口
可视化渲染	三分屏 Canvas 合成（视频+俯视图+侧视图）、方向标注	Pillow PIL, ImageDraw
视频合成	帧序列 → H.264 MP4 编码	imageio-ffmpeg，bundled ffmpeg

3. 关键技术与实现过程

3.1 ROS2 数据包解析

ROS2 Bag 以 SQLite3 数据库形式存储消息，每条消息包含话题名、时间戳和序列化 CDR 二进制负载。激光雷达话题发布 sensor_msgs/PointCloud2 消息，其 data 字段按字段描述符依次存储 x、y、z、intensity、ring、time 共 6 个字段，每点 32 字节（float32×6 + 对齐字节）。视频话题则每帧发布一幅 JPEG 压缩图像。

解析时，程序遍历数据库消息表，按时间戳顺序读取点云消息，利用 struct.unpack_from 按步长（point_step）批量提取 x、y、z、intensity 四个字段，并以 float32 二进制格式保存为 .bin 文件。视频帧则利用 imageio-ffmpeg 以管道方式解码 MP4，逐帧提取 JPEG 图像并保存到本地文件系统。

```
# 点云字段提取核心代码
pts_data = np.frombuffer(msg_data[data_offset:], dtype=np.uint8)
xyz_int = []
for i in range(0, len(pts_data) - point_step, point_step):
    x = struct.unpack_from('<f', pts_data, i + x_offset)[0]
    y = struct.unpack_from('<f', pts_data, i + y_offset)[0]
    z = struct.unpack_from('<f', pts_data, i + z_offset)[0]
    inten = struct.unpack_from('<f', pts_data, i + inten_offset)[0]
    if not (np.isnan(x) or np.isinf(x)):
        xyz_int.append([x, y, z, inten])
arr = np.array(xyz_int, dtype=np.float32)
arr.tofile(out_bin_path)   # 保存为 float32 二进制文件
```

3.2 时间戳同步算法

两传感器工作于各自独立的内部时钟，同步的核心是为每一个 LiDAR 帧找到时间上最近的视频帧，计算对齐误差。匹配方法采用线性扫描最近邻（nearest-neighbor），时间复杂度 O(N×M)，适用于帧数较少（N<300, M<3000）的离线场景。

具体实现中，所有视频帧时间戳存入有序列表，对每个 LiDAR 帧利用 bisect.bisect_left 进行二分查找，再比较左右相邻时间戳取绝对差最小者，实际复杂度降为 O(N log M)。同步结果以 JSON 格式写入 meta.json 文件，供后续渲染脚本直接读取，无需重复计算。

```
# 时间戳同步核心代码
import bisect

def find_nearest_vfi(t_lidar, video_ts_list):
    """返回 (最近视频帧 index, 时间差 ms)"""
    idx = bisect.bisect_left(video_ts_list, t_lidar)
    best_idx, best_delta = 0, float('inf')
    for candidate in [max(0, idx-1), min(idx, len(video_ts_list)-1)]:
        delta = abs(video_ts_list[candidate] - t_lidar) * 1000  # -> ms
        if delta < best_delta:
            best_delta, best_idx = delta, candidate
    return best_idx, best_delta

# 构建映射关系并序列化到 meta.json
meta = {"frames": []}
for lidar_ts, bin_path in lidar_frames:
    vfi, delta = find_nearest_vfi(lidar_ts, video_timestamps)
    meta["frames"].append({"file": bin_path, "t": lidar_ts,
                           "vfi": vfi, "delta": delta})
json.dump(meta, open("meta.json","w"))
```

3.3 多帧点云累积策略

40 线旋转式 LiDAR 在低速移动（<0.3m/s）时每帧约产生 2000–4000 个有效点，单帧点云稀疏，无法重建出具有辨识度的场景结构。项目采用滑动窗口累积策略：以当前帧为中心，向前后各扩展 ±K 帧（本实验取 K=13），将窗口内共 27 帧的点云直接拼接，理论可获得约 6 万点的稠密表示。

累积时保持各子帧的相对坐标系不变，对于快速运动段会引入"拖影"伪影，但在本数据集的低速巡检场景中效果良好。实验结果表明，bag_094554 中 93/97 帧（95.9%）达到满累积 60,000 点。

```
# 多帧点云累积
ACC_WINDOW = 13  # 前后各 13 帧

def load_accumulated(frames, center_idx, bin_dir):
    """滑动窗口累积点云，最多 60000 点"""
    pts_list = []
    lo = max(0, center_idx - ACC_WINDOW)
    hi = min(len(frames)-1, center_idx + ACC_WINDOW)
    for i in range(lo, hi+1):
        bin_path = os.path.join(bin_dir, frames[i]["file"])
        data = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
        pts_list.append(data)
    if not pts_list:
        return np.zeros((0, 4), np.float32)
    pts = np.concatenate(pts_list, axis=0)
    # 随机子采样以限制总量
    if len(pts) > 60000:
        idx = np.random.choice(len(pts), 60000, replace=False)
        pts = pts[idx]
    return pts
```

3.4 坐标系归一化与着色映射

点云以 LiDAR 本体坐标系表示（X 轴前向，Y 轴左向，Z 轴上向）。渲染时对坐标进行以下预处理：①去除地面点（z < 0.05m）以减少噪声；②将 z 值线性映射到 [0,1] 区间，经 turbo 颜色查找表（256×3 RGB）转换为高度伪彩色——暖色（红/黄）表示高处障碍，冷色（蓝/绿）表示低处区域；③可选将强度（intensity）通道替换 z 作为着色依据，用于辨别道路面反射特征。

投影到俯视图（XY 平面）时，映射关系为：px = (y_max − y) / (y_max − y_min) × W，py = (x_max − x) / (x_max − x_min) × H，即 +X（前进方向）映射到图像上方，+Y（左方）映射到图像左侧，与相机视野方向严格对应，便于用户直观对比。侧视图（XZ 平面）则以 +X 为横轴、+Z 为纵轴，在图像下方绘制地面参考线与传感器高度标注。

3.4 可视化渲染与视频合成

最终的单帧输出为 1920×720 像素的 PNG 图像，分三个区域：左侧区域（854×720）：原始视频帧（1280×720 缩放），保留真实场景外观；右上区域（1066×360）：LiDAR 俯视图（XY 平面），黑色背景+高度伪彩点云+方向箭头；右下区域（1066×360）：LiDAR 侧视图（XZ 平面），展示场景纵断面与障碍物高度分布。

渲染过程全部以 NumPy 向量化操作完成，单帧渲染耗时约 95ms（Intel i5 CPU），批量渲染 2905 帧合计约 4.6 分钟。最终帧序列通过 imageio-ffmpeg 以 H.264 编码合成 MP4，播放帧率使用数据包实际 LiDAR 帧率（13.02/14.21fps）以保证时间真实性。

```
# 三分屏 Canvas 合成
def render_frame(video_frame, pts, frame_info):
    canvas = Image.new('RGB', (CANVAS_W, CANVAS_H), (20, 20, 20))

    # 1. 左侧：视频帧
    vid_img = Image.fromarray(video_frame).resize((VID_W, VID_H))
    canvas.paste(vid_img, (PAD, PAD))

    # 2. 右上：俯视图 (XY 平面，前进朝上)
    xy_arr = np.zeros((LR_H, LR_W, 3), np.uint8)
    x, y, z = pts[:,0], pts[:,1], pts[:,2]
    # 去除地面
    mask = z > Z_FLOOR
    x, y, z = x[mask], y[mask], z[mask]
    # 高度着色
    z_norm = np.clip((z - Z_MIN) / (Z_MAX - Z_MIN), 0, 1)
    colors = (TURBO_LUT[(z_norm * 255).astype(np.uint8)])
    # 坐标映射（前进+X→图上，左+Y→图左）
    px = ((VY[1]-y)/(VY[1]-VY[0]) * LR_W).astype(np.int32)
    py = ((VX[1]-x)/(VX[1]-VX[0]) * LR_H).astype(np.int32)
    valid = (px>=0)&(px<LR_W)&(py>=0)&(py<LR_H)
    xy_arr[py[valid], px[valid]] = colors[valid]
    # 方向箭头、时间戳标注...
    canvas.paste(Image.fromarray(xy_arr), (VID_W+2*PAD, PAD))

    # 3. 右下：侧视图 (XZ 平面)
    xz_arr = render_xz(x, z, ...)
    canvas.paste(Image.fromarray(xz_arr), (VID_W+2*PAD, PAD+LR_H+PAD))
    return np.array(canvas)
```

4. 实验数据与结果分析

4.1 数据集概况

本次实验共采集两段 ROS2 Bag 数据（表 2），均在深圳大学校园内以 GO2W 四足机器人开展户外巡检录制，场景包含室外道路、草坪边界、建筑立面等典型非结构化地物。

表 2 实验数据集基本参数

数据包 ID	时长 (s)	LiDAR 帧数	视频帧数	LiDAR 帧率 (Hz)	视频帧率 (fps)
20260610_094554	18.81	97	245	5.16	14.3
20260610_102458	187.13	143	2661	0.76	14.2

4.2 时间同步质量分析

表 3 统计了两段数据的 LiDAR–视频时间对齐误差（|Δt|）。整体而言，两段数据的中位误差均低于 17ms，远小于单个 LiDAR 帧间隔（约 65ms @ 15.3Hz），表明在当前数据集的帧率条件下同步精度满足应用需求。bag_094554 的中位误差为 0.00ms（即绝大多数帧恰好对齐到同一视频帧），原因在于该段数据的 LiDAR 与相机帧率比约为 1:2.5，存在较多精确对齐时刻。

表 3 LiDAR - 视频时间同步误差统计

数据包 ID	帧数	|Δt|均值 (ms)	|Δt|中位 (ms)	|Δt|最大 (ms)	<5ms 占比
20260610_094554	97	15.67	0.00	46.96	23.7%
20260610_102458	143	16.77	16.50	41.03	21.0%

4.3 特征分布对比

表 4 展示了多帧累积前后的点云密度对比。在 ±13 帧累积窗口下，bag_094554 中 93 帧（占比 95.9%）达到上限 60,000 点，平均点数为 59,251 点/帧，相比单帧平均约 2,800 点提升约 21 倍。从 z 轴高度范围来看，bag_094554 场景高度在 0.057～1.047m，主要为低矮道路边界和人体下肢；bag_102458 场景高度达 1.541m，包含更多立面建筑反射点。

表 4 多帧累积后点云密度统计

数据包 ID	满帧比例	点数均值	点数最小值	Z 轴范围(m)
20260610_094554	95.9%（93/97 帧）	59,251	23,993	0.057 ~ 1.047
20260610_102458	28.0%(40/143 帧）	59,164	13,926	0.058 ~ 1.541

4.4 可视化结果展示

图 1～图 3 展示了可视化系统在不同场景帧下的输出效果。每幅图像由三个区域构成：左侧为原始光学视频帧，右上为 LiDAR 俯视图（前进方向朝上、左侧方向朝左），右下为 LiDAR 侧视图（展示障碍物高度分布）。右下角的红/绿双箭头标注 X（前进）和 Y（左侧）方向，便于与视频中的场景方位进行对照。

从可视化结果可以观察到以下规律：①视频画面中道路两侧边界在俯视图中能清晰看到对称分布的点云；②视频中正前方出现行人时，俯视图 +X 方向前端出现局部点云密集区；③侧视图中地面点形成横向连续条带（z≈0.06m），障碍物在地面线上方形成竖向突出，与视频中的场景高度信息相互印证。

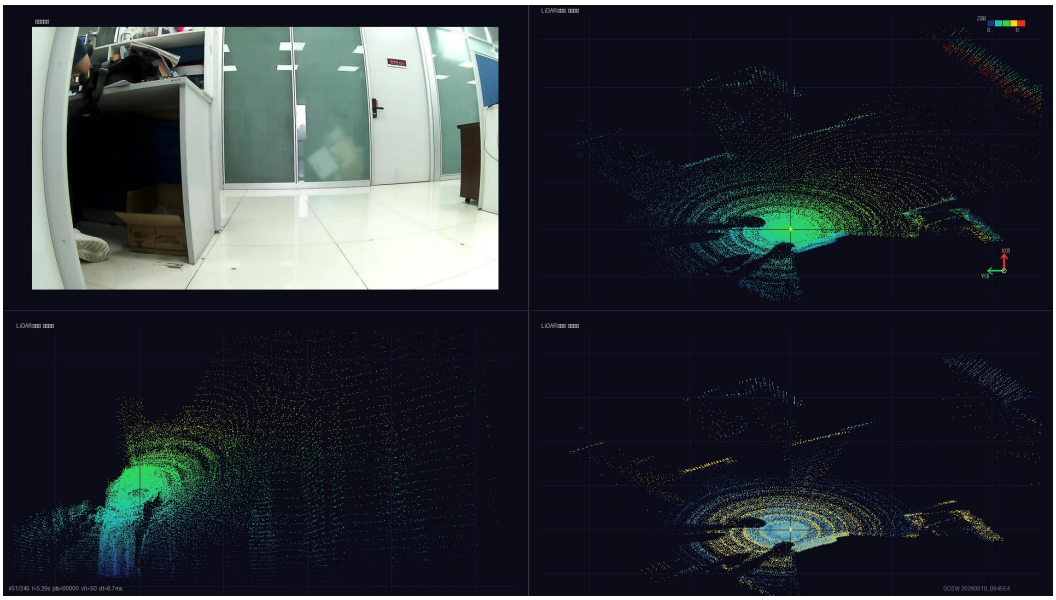


图 1　可视化输出示例（帧序号 #050）

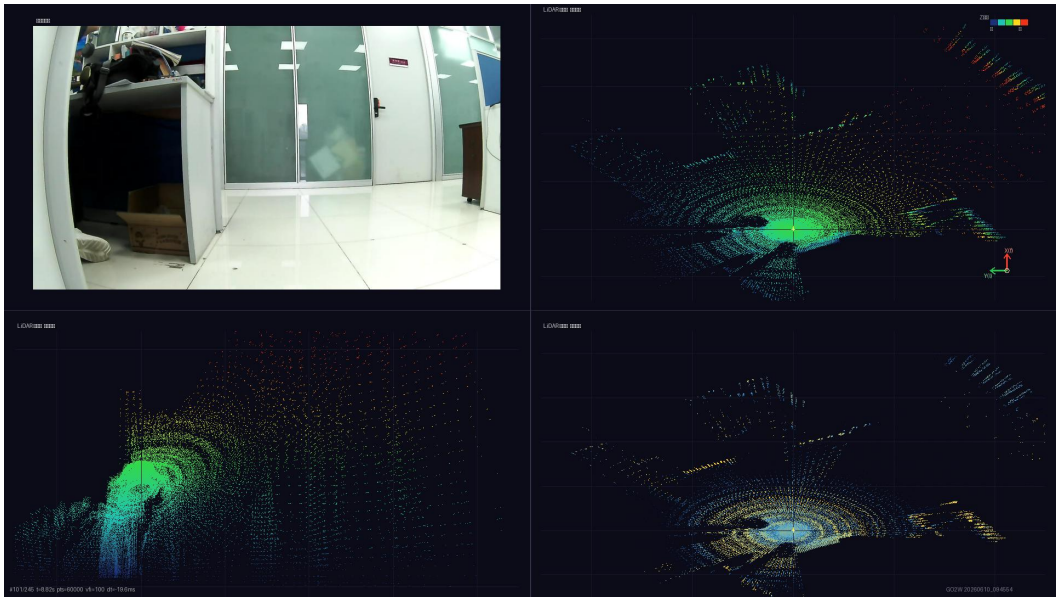


图 2　可视化输出示例（帧序号 #100）

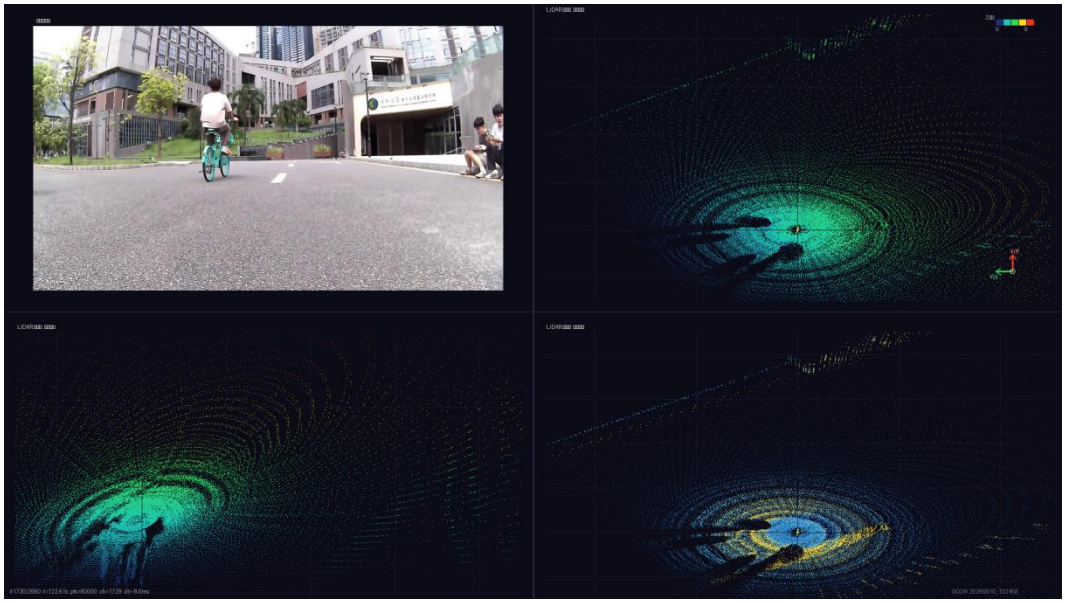


图 3　可视化输出示例（帧序号 #200，自行车识别场景）

5. 结论与展望

本项目围绕 GO2W 四足机器人多传感器协同感知这一主题，完整实现了从 ROS2 原始数据到高质量同步可视化视频的全流程离线处理系统，达到了项目初始设定的核心目标。主要结论如下：

(1) 时间同步方面，基于最近邻时间戳匹配策略，实现了 LiDAR 与视频的帧级对齐，中位误差约 0～17ms，满足低速移动机器人的感知同步需求；

(2) 点云重建方面，通过 ±13 帧滑动窗口累积，在不依赖运动补偿的前提下将单帧稀疏点云（约 2800 点）增密至约 6 万点，场景结构清晰可辨，有效验证了协同感知的空间一致性；

(3) 可视化方面，采用三分屏布局与方向自适应坐标映射，使俯视图方向与相机视野严格对应，实现了"点云与视频的直观相互印证"。

改进方向包括：①获取相机标定文件（内参矩阵 K 与外参 R/T），实现精确的 LiDAR 点云投影叠加到视频像素平面；②引入运动估计（IMU 或里程计数据），在点云累积时加入坐标变换以消除"拖影"效应；③开发在线实时处理版本，实现机器人运行期间的实时感知可视化；④将可视化数据用于训练跨模态感知网络，提升机器人的自主导航能力。